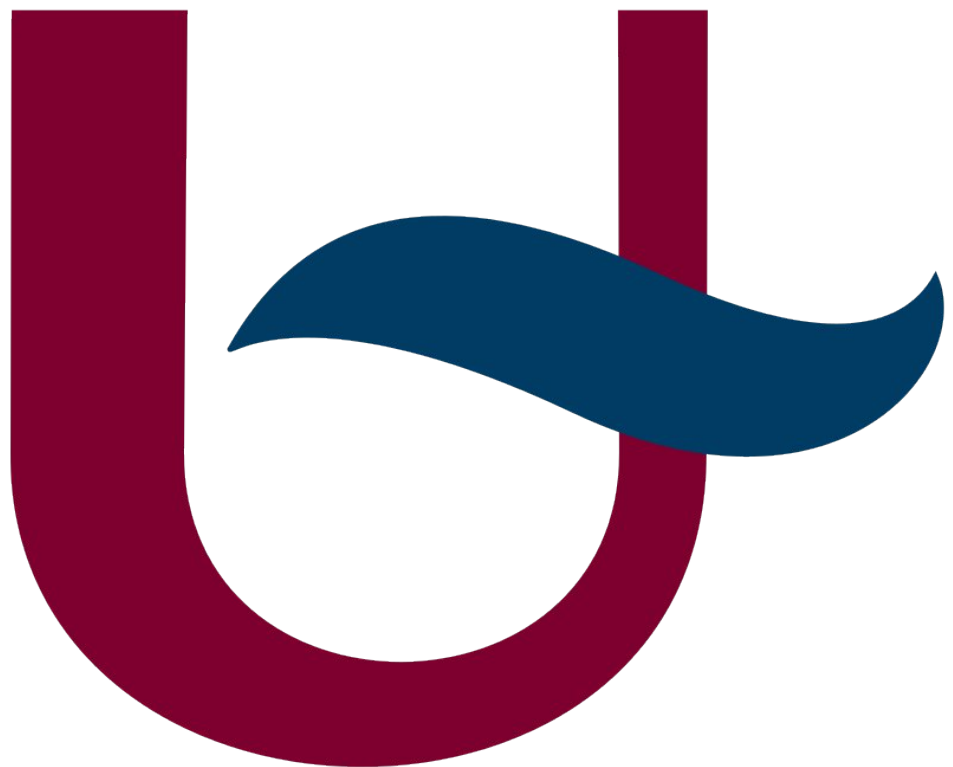


I-MAP: LOGBOOK (AI USAGE)

TSMK



LLM request

△ [ChatGPT](#) was used for this prompt.

install platform.io

LLM response

To install **PlatformIO**, the recommended way is through **Visual Studio Code** as an extension.

✓ Option 1 (Recommended): Install PlatformIO in VS Code

1 Install Visual Studio Code

Download and install from:

<https://code.visualstudio.com/>

2 Install PlatformIO Extension

1. Open **Visual Studio Code**
2. Click the **Extensions** icon (left sidebar)
3. Search for **PlatformIO IDE**
4. Click **Install**

The extension will automatically install:

- PlatformIO Core
- Python (if needed)
- Required toolchains

✓ Option 2: Install PlatformIO Core (CLI only)

If you prefer command line:

◆ Windows (PowerShell)

```
pip install -U platformio
```

◆ macOS / Linux

```
pip3 install -U platformio
```

After installation, verify:

```
pio --version
```

⚡ If pip is not installed

Install Python first from:

<https://python.org>

Then ensure pip works:

```
python -m ensurepip --upgrade
```

🔥 Alternative: Install via Homebrew (macOS)

If you use **Homebrew**:

```
brew install platformio
```

If you tell me:

- Your **OS** (Windows / macOS / Linux)
- Whether you want **GUI (VS Code)** or **CLI only**

I can give you step-by-step instructions tailored to your setup.

LLM request

I'm using debian 12 and have installed it via vs-code plugins. When i try "pio boards" in the command prompt it sais "bash: pio: command not found".

LLM response

This is expected behavior 🤔 — when you install PlatformIO via **Visual Studio Code**, the pio command is **not automatically added to your system PATH**.

The extension uses its own internal PlatformIO Core.

✓ Solution Options (Debian 12)

◆ Option 1 (Easiest): Use PlatformIO inside VS Code

Instead of `pio boards`, use:

1. Open VS Code
2. Open Command Palette (Ctrl + Shift + P)
3. Type:
4. PlatformIO: Boards

Or open the PlatformIO Home tab → Boards.

No terminal needed.

◆ Option 2: Use PlatformIO Core from VS Code's internal location

PlatformIO installed by the extension usually lives here:

`~/.platformio/penv/bin/pio`

Try:

`~/.platformio/penv/bin/pio boards`

If that works, then your issue is just PATH.

◆ Option 3 (Recommended if you want CLI access):

Add it to PATH

Edit your `.bashrc`:

`nano ~/.bashrc`

Add this line at the bottom:

`export PATH="$HOME/.platformio/penv/bin:$PATH"`

Save and reload:

`source ~/.bashrc`

Now test:

`pio --version`

`pio boards`

◆ Option 4 (Cleanest CLI setup): Install PlatformIO

Core globally

If you want full CLI independence from VS Code:

`pip3 install -U platformio`

Then:

`pio --version`

🧠 Why This Happens

The VS Code extension installs:

- PlatformIO Core
- A private Python virtual environment

It does **not** modify your system shell configuration (which is actually good practice).

If you want, tell me:

- Do you want to use PlatformIO mainly inside VS Code?
- Or do you plan to automate builds via terminal/CI?

I can recommend the best long-term setup for Debian.

LLM request

How can i check all connected boards

LLM resopnse

On **Debian 12**, there are two slightly different things you might mean by “connected boards”:

1. Boards **physically connected via USB**
2. Boards **supported by PlatformIO**

I'll show both.

1 Check physically connected boards (USB devices)

◆ List USB devices

```
lsusb
```

You should see something like:

```
Bus 001 Device 005: ID 2341:0043 Arduino SA Uno R3
```

If you're using something like an ESP32, you may see:

- Silicon Labs CP210x
- CH340
- FTDI
- etc.

◆ Check available serial ports

Most dev boards show up as a serial device:

```
ls /dev/ttyUSB*
```

```
ls /dev/ttyACM*
```

Or more reliably:

```
dmesg | grep tty
```

After plugging in the board.

2 Using PlatformIO to detect connected boards

Inside terminal (if pio works):

```
pio device list
```

This shows connected serial devices.

Or the more detailed command:

```
pio device monitor
```

3 Inside VS Code (if CLI isn't configured)

In **Visual Studio Code**:

1. Open Command Palette (Ctrl + Shift + P)
2. Run:
3. PlatformIO: List Serial Ports

Or open:

PlatformIO Home → Devices

If Your Board Doesn't Show Up

Common causes on Debian:

◆ Missing permissions (very common)

Add yourself to the dialout group:

```
sudo usermod -a -G dialout $USER
```

Then **log out and log back in**.

◆ Bad USB cable

Some USB cables are power-only (no data).

◆ Driver issues

Most Arduino/ESP boards work out of the box on Debian 12, but if it's a CH340-based board, you can check:

```
lsmod | grep ch34
```

If You Tell Me:

- Which board model?
- Does `lsusb` show anything?
- Does `/dev/ttyUSB0` or `/dev/ttyACM0` appear?

I can diagnose it step-by-step.

LLM request

pio device list worked

How can i upload to that board

LLM response

Great 🙌 since `pio device list` works, you're ready to upload.

Uploading with PlatformIO requires a **project** (you can't upload a random `.ino` file directly like in the Arduino IDE).

✓ Step 1 — Create a Project

Inside a working directory:

```
mkdir my_project
cd my_project
pio project init --board <board_id>
```

You need the correct `<board_id>`.

To find it:

```
pio boards
```

Example:

- Arduino Uno → `uno`
- ESP32 Dev Module → `esp32dev`
- ESP8266 NodeMCU → `nodemcu2`

Example:

```
pio project init --board uno
```

✓ Step 2 — Add Your Code

Edit:

```
src/main.cpp
```

Example test:

```
#include <Arduino.h>
```

```
void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
}
```

```
void loop() {
  digitalWrite(LED_BUILTIN, HIGH);
  delay(500);
  digitalWrite(LED_BUILTIN, LOW);
  delay(500);
}
```

✓ Step 3 — Upload to the Board

Now simply run:

```
pio run --target upload
```

PlatformIO will:

1. Compile
2. Detect the serial port
3. Upload automatically

🔧 If Auto-detection Fails

Find your port:

```
pio device list
```

Example output:

```
/dev/ttyUSB0
```

Then upload with explicit port:

```
pio run --target upload --upload-port /dev/ttyUSB0
```



Optional: Add Upload Port

Permanently

Edit `platformio.ini`:

```
upload_port = /dev/ttyUSB0
```



Quick Alternative (Inside VS Code)

If using **Visual Studio Code**:

Just click the **Upload** → **arrow button** in the bottom bar.

If you tell me your exact board model, I can give you the exact board ID and a working minimal config.

LLM request

The teensy does blink but the upload states there is a problem

```
pio run -t upload --upload-port /dev/ttyACM0
Processing teensy41 (platform: teensy; board: teensy41; framework: arduino)
-----
-----
Verbose mode can be enabled via -v, --verbose option
CONFIGURATION: https://docs.platformio.org/page/boards/teensy/teensy41.html
PLATFORM: Teensy (5.0.0) > Teensy 4.1 HARDWARE: IMXRT1062 600MHz,
512KB RAM, 7.75MB Flash
DEBUG: Current (jlink) External (jlink)
PACKAGES:
- framework-arduinoteensy @ 1.159.0 (1.59)
- tool-teensy @ 1.159.0 (1.59)
- toolchain-gccarmnoneabi-teensy @ 1.110301.0 (11.3.1)
LDF: Library Dependency Finder -> https://bit.ly/configure-pio-ldf
LDF Modes: Finder ~ chain, Compatibility ~ soft Found 92 compatible libraries
Scanning dependencies...
No dependencies
Building in release mode
Linking .pio/build/teensy41/firmware.elf
Checking size .pio/build/teensy41/firmware.elf
Advanced Memory Usage is available via "PlatformIO Home > Project Inspect"
Building .pio/build/teensy41/firmware.hex Configuring upload protocol...
AVAILABLE: jlink, teensy-cli, teensy-gui
CURRENT: upload_protocol = teensy-gui
Uploading .pio/build/teensy41/firmware.hex
Opening Teensy Loader...
error sending reboot command to /dev/hidraw3
```

LLM resopnse

Perfect — this error tells us exactly what's wrong 🙌

You're using a **Teensy 4.1**, and Teensy boards **do NOT upload over /dev/ttyACM0** like Arduino boards.

They upload via **USB HID**, not serial.

Your error:

error sending reboot command to /dev/hidraw3

means **Linux permission issue on the HID device**.

✓ ✓ **FIX (Debian 12)**

You must add proper **udev rules** for Teensy.

◆ **Step 1 — Create udev rule**

```
sudo nano /etc/udev/rules.d/49-teensy.rules
```

Paste this:

```
SUBSYSTEM=="usb", ATTR{idVendor}=="16c0", MODE=="0666"
```

```
KERNEL=="hidraw*", ATTRS{idVendor}=="16c0", MODE=="0666"
```

Save and exit.

◆ **Step 2 — Reload udev**

```
sudo udevadm control --reload-rules
```

```
sudo udevadm trigger
```

◆ **Step 3 — Unplug and replug the Teensy**

Now try:

```
pio run -t upload
```

△ Remove --upload-port. It is NOT needed for Teensy.

🔍 **Why This Happens**

Teensy uses:

- Vendor ID: 16c0
- HID device: /dev/hidrawX

Linux blocks access to /dev/hidraw* by default for normal users.

The rule fixes that.

🔥 **Alternative (Quick Test)**

You can confirm it's a permission issue by trying:

```
sudo pio run -t upload
```

If that works → 100% udev permission problem.

But **don't use sudo permanently** — fix the rule instead.

🧠 **Also Important**

In your log:

```
CURRENT: upload_protocol = teensy-gui
```

You can switch to CLI uploader (more reliable on Linux):

Edit `platformio.ini`:
`upload_protocol = teensy-cli`
Then upload again.
This avoids the graphical Teensy Loader.

After This

It should upload instantly without pressing the reset button.
If it still fails, send:
`ls -l /dev/hidraw*`
and I'll diagnose it precisely.
erfect — this error tells us exactly what's wrong 🍵

